

자료구조 개요

1. 자료구조의 개념

자료구조는 **단순 구조**, **선형 구조**, **비선형 구조**, **파일 구조**로 나뉜다. 본 강의에서는 먼저 **단순 구조**에 대한 데이터 표현 방식을 학습한다.

2. 데이터 표현 방식

데이터 표현 방식은 **수치 자료**, **문자 자료**, **논리 자료**, **포인터 자료**, **문자열 자료**로 나뉜다.

2.1 수치 자료

- **10진수 표현**
 - ☒ **존(Zone) 형식**: 인쇄용 문자 표현 방식
 - ☒ **팩(Packed) 형식**: 숫자 표현 방식
- **2진수 표현**
 - ☒ **정수 표현**: 부호-절대값, 1의 보수, 2의 보수 방식
 - ☒ **실수 표현**: 고정 소수점, 부동 소수점 방식

2.2 문자 자료

- **BCD 코드(6비트)**: 앞 2비트는 존 비트, 뒤 4비트는 디지털 비트
- **EBCDIC 코드**: IBM에서 주로 사용
- **ASCII 코드**: 공용 컴퓨터에서 일반적으로 사용

2.3 논리 자료

논리 자료는 참(True)과 거짓(False)을 표현하는 자료형이다.
주로 **1바이트의 최하위 비트**로 표현되며, 방식은 다음과 같다.

1. 최하위 비트가 1이면 참, 0이면 거짓
2. 전체 비트가 1이면 참, 0이면 거짓
3. 하나 이상의 비트가 1이면 참, 모두 0이면 거짓

2.4 포인터 자료

- 메모리의 **주소를 표현하는 자료 형식**
- 포인터는 **주소와 값**으로 구성됨
- 하드디스크의 구조(트랙과 섹터)를 기반으로 저장 위치를 가짐

2.5 문자열 자료

- 문자가 연속된 **집합**
- **저장 방식**
 1. **구분자 방식**: 부분 문자열 사이에 구분자를 뒤서 저장
 2. **고정 길이 방식**: 일정한 길이로 저장 (메모리 낭비 발생 가능)

3. 포인터 방식: 헤더를 두고 포인터를 이용해 문자열 위치 지정

🔍 3. 자료구조와 알고리즘

- 자료구조 = 데이터 + 연산 알고리즘
- 예: 배열(데이터) + 순차 탐색(알고리즘) = 탐색 프로그램

💡 3.1 알고리즘 개념

알고리즘은 **컴퓨터로 문제를 해결하기 위한 단계적 절차**이며, 다음 5가지 조건을 만족해야 한다.

1. **입력(Input)**: 0개 이상의 입력이 존재해야 함
2. **출력(Output)**: 1개 이상의 출력이 반드시 존재해야 함
3. **명확성(Definiteness)**: 명령이 명확해야 함
4. **유한성(Finiteness)**: 반드시 종료해야 함
5. **유효성(Effectiveness)**: 실행 가능한 연산이어야 함

⚙️ 3.2 알고리즘 개발 과정

1. 문제 정의 → 요구사항 분석
 2. 알고리즘 작성 → 데이터 구조 지원
 3. 프로그램 작성 및 번역 (컴파일러/인터프리터)
 4. 실행 및 결과 확인
-

📄 4. 알고리즘 표현 방식

4.1 📊 흐름도 (Flowchart)

- 시작/끝, 입출력, 연산/처리, 조건 판단 등을 도식화

4.2 ✎ 유사 코드 (Pseudo Code)

- 프로그램 작성 전, 사람이 이해하기 쉽게 알고리즘을 표현

4.3 🖨 프로그래밍 언어

- C, Python 등 실제 코드로 구현
-

📦 5. 추상 데이터 타입 (ADT)

- 데이터와 연산을 정의하지만 구현 방법은 지정하지 않는 개념
 - ****객체(Object)와 연산(Operations)****으로 구성됨
 - ADT는 **정보 은닉**을 통해 사용자가 내부 구현을 알 필요 없게 함
-

⌚ 6. 알고리즘 성능 분석

6.1 ⌚ 시간 복잡도(Time Complexity)

- 실행 시간을 분석하는 방법
- 측정 방식
 1. **Clock Pulse 활용**: CPU 단위로 측정
 2. **Timer 함수 활용**: 초 단위로 측정

6.2 공간 복잡도(Space Complexity)

- 알고리즘이 사용하는 **메모리 크기** 분석

 7. 알고리즘의 점근적 표기법

- **Big-O(빅오, O)**: 최악의 성능
- **Big-Ω(빅오메가, Ω)**: 최상의 성능
- **Big-Θ(빅세타, Θ)**: 평균적인 성능

7.1 Big-O 표기법 예시

알고리즘 유형	Big-O 표기법
상수 시간	$O(1)$
로그 시간	$O(\log n)$
선형 시간	$O(n)$
선형 로그 시간	$O(n \log n)$
2차 시간	$O(n^2)$
지수 시간	$O(2^n)$
팩토리얼 시간	$O(n!)$

 8. 순환(재귀, Recursion)

8.1 재귀 개념

- 자기 자신을 호출하는 함수
- 문제를 작은 단위로 나누어 해결하는 방식

8.2 재귀와 반복 비교

- **재귀**: 자기 자신을 호출해 문제 해결
- **반복**: 루프(while, for)를 사용해 문제 해결

8.3 대표적인 재귀 알고리즘 예시

- 팩토리얼(Factorial)
- 피보나치 수열(Fibonacci Sequence)
- 하노이 탑(Tower of Hanoi)
- 이진 탐색(Binary Search)

🎯 9. 결론

자료구조와 알고리즘은 **프로그램 성능을 결정하는 핵심 요소**이다. 본 강좌에서는 다양한 자료구조와 알고리즘을 병행하여 학습하며, **효율적인 문제 해결 방법**을 익힐 것이다.